

17 juin 2026

CI/CD

Docker Compose & Registries

03/

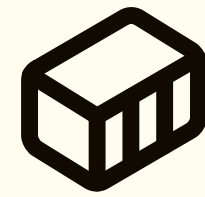
09

Ynov Campus Toulouse
M1 Expert en Développement Full Stack



9 séances

01	Intro DevOps & CI/CD
02	Conteneurisation Docker
03	Docker Compose + Registry
04	Tests automatisés + qualité
05	Infrastructure & sécurité
06	Déploiement automatisé
07	Monitoring & observabilité
08	Optimisation & prép soutenance
09	Soutenance



Là où on s'est arrêté

01 / 09



Ce qu'on maîtrise

Concepts

Images & conteneurs

Dockerfile

Commandes de base

En pratique

Conteneurisé une image

Build & run



Les limites

- **Lancement manuel de chaque conteneur**
- Pas de réseau entre conteneur par défaut
- Ordre de démarrage géré à la main
- Pas de persistance de données
- Config répétitive
- Impossible à partager simplement



Les limites

- Lancement manuel de chaque conteneur
- Pas de réseau entre conteneur par défaut
- Ordre de démarrage géré à la main
- Pas de persistance de données
- Config répétitive
- Impossible à partager simplement



Les limites

- Lancement manuel de chaque conteneur
- Pas de réseau entre conteneur par défaut
- Ordre de démarrage géré à la main
- Pas de persistance de données
- Config répétitive
- Impossible à partager simplement



Les limites

- Lancement manuel de chaque conteneur
- Pas de réseau entre conteneur par défaut
- Ordre de démarrage géré à la main
- Pas de persistance de données
- Config répétitive
- Impossible à partager simplement



Les limites

- Lancement manuel de chaque conteneur
- Pas de réseau entre conteneur par défaut
- Ordre de démarrage géré à la main
- Pas de persistance de données
- Config répétitive
- Impossible à partager simplement



Les limites

- Lancement manuel de chaque conteneur
- Pas de réseau entre conteneur par défaut
- Ordre de démarrage géré à la main
- Pas de persistance de données
- Config répétitive
- Impossible à partager simplement



Objectif du jour

- **Docker Compose** - orchestrer avec un seul fichier
- Réseaux - faire communiquer les conteneurs
- Volumes - persister les données
- Registries - partager les images
- Multi-stage - des images légères
- CI - tout automatiser



Objectif du jour

- **Docker Compose** - orchestrer avec un seul fichier
- **Réseaux** - faire communiquer les conteneurs
- **Volumes** - persister les données
- **Registries** - partager les images
- **Multi-stage** - des images légères
- **CI** - tout automatiser



Objectif du jour

- **Docker Compose** - orchestrer avec un seul fichier
- **Réseaux** - faire communiquer les conteneurs
- **Volumes** - persister les données
- Registries - partager les images
- Multi-stage - des images légères
- CI - tout automatiser



Objectif du jour

- **Docker Compose** - orchestrer avec un seul fichier
- **Réseaux** - faire communiquer les conteneurs
- **Volumes** - persister les données
- **Registries** - partager les images
- **Multi-stage** - des images légères
- **CI** - tout automatiser



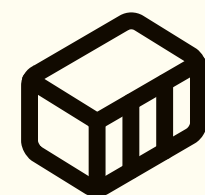
Objectif du jour

- **Docker Compose** - orchestrer avec un seul fichier
- **Réseaux** - faire communiquer les conteneurs
- **Volumes** - persister les données
- **Registries** - partager les images
- **Multi-stage** - des images légères
- **CI** - tout automatiser



Objectif du jour

- **Docker Compose** - orchestrer avec un seul fichier
- **Réseaux** - faire communiquer les conteneurs
- **Volumes** - persister les données
- **Registries** - partager les images
- **Multi-stage** - des images légères
- **CI** - tout automatiser



Docker Compose

02 / 09

Orchestrer une application multi-conteneurs avec un seul
fichier déclaratif.



C'est quoi ?

→ Définir & exécuter des apps multi-conteneurs

→ Un fichier YAML

→ Une commande pour tout démarrer



C'est quoi ?

- Définir & exécuter des apps multi-conteneurs
- Un fichier YAML
- Une commande pour tout démarrer



C'est quoi ?

- Définir & exécuter des apps multi-conteneurs
- Un fichier YAML
- Une commande pour tout démarrer



10 commandes → 1 fichier

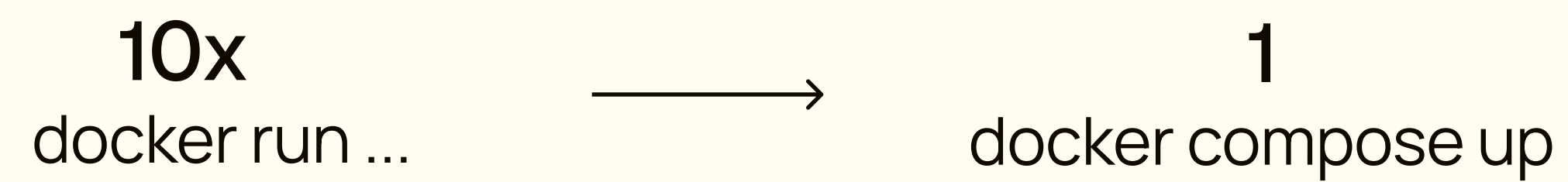
10x
docker run ...



1
docker compose up



10 commandes → 1 fichier





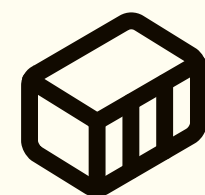
Docker-compose.yml

```
1  services:
2    frontend:
3      image: mon-frontend:latest
4      ports:
5        - "3000:3000"
6
7    backend:
8      image: mon-backend:latest
9      ports:
10       - "5000:5000"
```



Docker-compose.yml

```
1  services:
2    client:
3      build:
4        context: ./client
5        dockerfile: Dockerfile.dev
6      ports:
7        - '3000:3000'
8      env_file:
9        - ../.env
```

Réseaux Docker

03 / 09

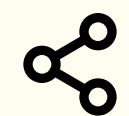
Comment des conteneurs isolés finissent par se parler



Isolés par défaut

- Chaque conteneur = un processus isolé
- Réseau isolé par défaut (sécurité)
- Ils ne peuvent pas se "voir" directement

... mais le frontend doit appeler le backend, et le backend la BDD.



Communiquer par nom

Le nom du service = nom d'hôte

A terminal window with a dark blue background and three colored window control buttons (red, yellow, green) at the top left. It contains a YAML configuration snippet for services.

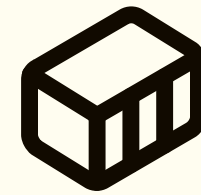
```
1  services:  
2    backend:  
3      environnement:  
4        - DB_HOST=database  
5    databse:  
6      image: postgres;15-alpine
```

postgresql://database:5432



Isoler avec plusieurs réseaux

```
1  services:
2    frontend:
3      networks: [frontend-net]
4    backend:
5      networks: [frontend-net, backend-net]
6    database:
7      networks: [backend-net]
8  networks:
9    frontend-net:
10   backend-net:
11
```



Volumes & persistance

04 / 09

Un conteneur est éphémère. Vos données, non.



Le problème de l'éphémérité

`docker run postgres`



INSERT données



`docker rm`



Données perdues



Le problème de l'éphémérité

`docker run postgres`



`INSERT données`



`docker rm`



Données perdues



Le problème de l'éphémérité

`docker run postgres`



`INSERT données`



`docker rm`



Données perdues



Le problème de l'éphémérité

`docker run postgres`



`INSERT données`



`docker rm`



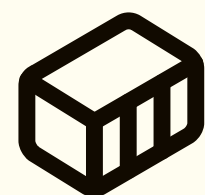
Données perdues



Persister une base



```
1  services:
2    database:
3      image: postgres:15-alpine
4      volumes:
5        - db-data:/var/lib/postgresql/data
6  volumes:
7    db-data:
```



Variables d'environnement

05 / 09



La mauvaise pratique

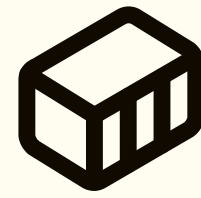


```
1  services:  
2    backend:  
3      environment:  
4        - DB_PASSWORD=supersecret
```



Externaliser dans .env

```
1 database:
2   image: postgres:13
3   ports:
4     - '${DATABASE_PORT}:5432'
5   env_file:
6     - ../.env
7   environment:
8     POSTGRES_USER: ${DATABASE_USER}
9     POSTGRES_PASSWORD: ${DATABASE_PASSWORD}
10    POSTGRES_DB: ${DATABASE_NAME}
11  volumes:
12    - pgdata:/var/lib/postgresql/data
```



Registries

06 / 09

Stocker , versionner et partager ses images Docker



Qu'est-ce qu'un registry ?

un serveur qui **stocke** et **distribue** les images

Publics

Docker Hub

ghcr.io

Privés

GitLab Registry

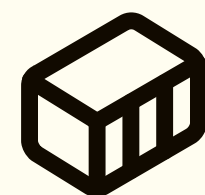
AWS ECR

Azure ACR



Versioning & tags

latest	dernière version - a éviter en prod
v1.0.0	version sémantique
commit-sha	traçabilité complète
prod / staging	par environnement



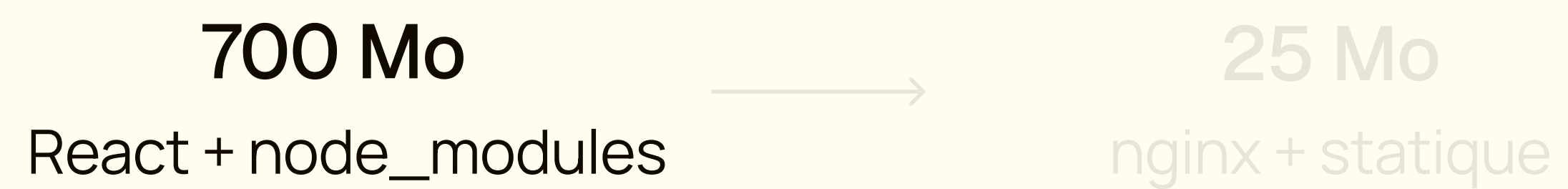
Multi-stage builds

06 / 08

Des images de production légères et sécurisées.

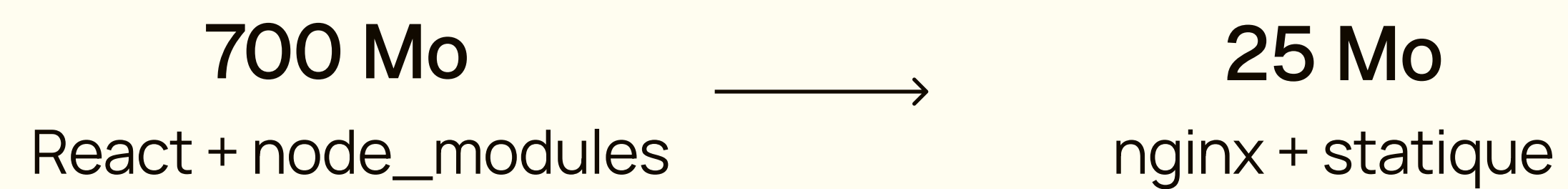


Le problème





Le problème

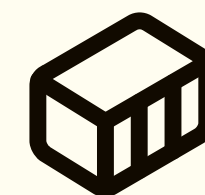




Un Dockerfile multi-stage



```
1  # Stage 1 - build (temporaire)
2  FROM node:22-alpine AS builder
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm i
6  COPY . .
7  RUN npm run build
8
9  # Stage 2 - production (image finale)
10 FROM nginx:alpine
11 COPY --from=builder /app/dist /usr/share/nginx/html
```



Docker dans GitLab CI

07 / 09

Builder et publier les images automatiquement
à chaque commit.



Le stage build



```
1  build_backend:
2    stage: build
3    image: docker:24
4    services: [docker:24-dind]
5    before_script:
6      - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
7    script:
8      - docker build -t $CI_REGISTRY_IMAGE/backend:$CI_COMMIT_SHORT_SHA .
9      - docker push $CI_REGISTRY_IMAGE/backend:$CI_COMMIT_SHORT_SHA
```




Le stage build

\$CI_REGISTRY	URL du registry GitLab
\$CI_REGISTRY_USER / _PASSWORD	identifiants auto
\$CI_REGISTRY_IMAGE	URL de l'image du projet
\$CI_COMMIT_SHORT_SHA	SHA du commit (7 car .)



Le problème de l'éphémérité

commit



GitLab CI - build



push → registry



tests / deploy



Le problème de l'éphémérité

commit



GitLab CI - build



push → registry



tests / deploy



Le problème de l'éphémérité

commit



GitLab CI - build



push → registry



tests / deploy



Le problème de l'éphémérité

commit



GitLab CI - build



push → registry



tests / deploy



Le problème de l'éphémérité

commit



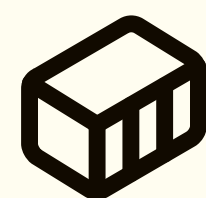
GitLab CI - build



push → registry



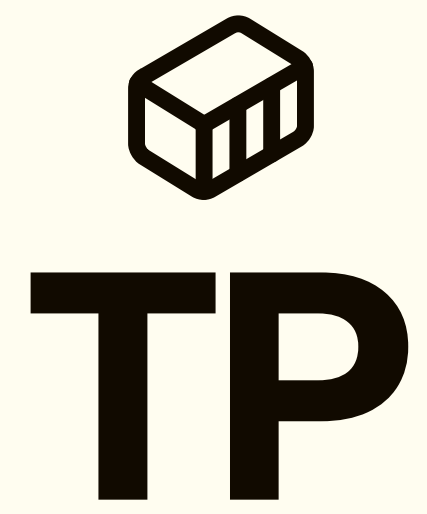
tests / deploy



Démo

09 / 10

Démo docker compose + pipeline CI



10 / 10

Builder et publier ses images



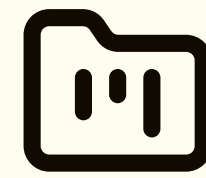
A rendre

- Monorepo backend + frontend
- Mutli stage
- docker-compose
- pipeline (build + push (+ tag))

Objet : NOM PRENOM - TP Docker Compose

Contenu : <https://hub.docker.com/u/username>
+ lien repo github (public)

thomas.fourties31@ynov.com



Mise en place du projet fil rouge



Mise en route

1. Former des groupes de 2-3 ou 4
2. Choisir une idée de projet
3. Choisir une stack technique
4. Créer un repo GitLab ou GitHub
5. Avoir un **README** sur le repo avant de partir



Les attendus

L'application

- Application fullstack : front + back + BDD
- Conteneurisée
- Stack libre

L'infrastructure

- Sécurisé (par clé SSH, user dédié ...)
- Orchestrateur

La chaîne CI/CD

- Stages (lint, test, build, deploy)
- Gestion des environnements

Monitoring

- Alertes
- Métriques système
- Logs des conteneurs